

Model-Driven Development of Web Application Backend Using Node.js Framework

Werayoot Kunphai

Department of Computer Engineering, Faculty of Engineering
Chulalongkorn University
Bangkok, Thailand
6570234721@student.chula.ac.th

Twittie Senivongse

Department of Computer Engineering, Faculty of Engineering
Chulalongkorn University
Bangkok, Thailand
twittie.s@chula.ac.th

Abstract—Model-Driven Development (MDD) has emerged as a promising software engineering paradigm that uses models as the primary artifacts to support software development including web application development. This paper presents a novel application of the MDD principles for Node.JS backend development, focusing on both forward and reverse engineering processes. A UML Profile for Node.js, supporting both SQL and NoSQL databases, is introduced for building design models of web applications. Transformation engines are provided for model-to-code (M2C) and code-to-model (C2M) transformation. The proposed approach was evaluated through three case studies. The forward engineering results demonstrated high model-to-code transformation rates especially for Model (86.18-94.53%) and Route (70.73-90.52%), with consistent performance across SQL (MySQL) and NoSQL (MongoDB) implementations. The reverse engineering results showed 100% correctness in code-to-model transformation for different implementations of backend code. This approach significantly reduces development time for code development while maintaining system documentation through automated model generation, providing a practical solution for Node.JS backend development and maintenance.

Keywords—model-driven development, Node.js, UML profile, forward engineering, reverse engineering

I. INTRODUCTION

Node.js [1] becomes the leading framework for web application backend development, capturing 42% market share according to Stack Overflow [2]. Expanding web backends face challenges including code redundancy, increased development time, and substantial onboarding efforts for new team members. These issues necessitate solutions for streamlined feature development, sustainable maintenance, and efficient knowledge transfer through centralized documentation. The concept of Model-Driven Development (MDD) has emerged as a promising paradigm that addresses these issues in software development, including web development, by using software models as the primary artifacts to drive software development activities, especially design and implementation. MDD automates the transformation of models from one form into another, where each model, both source and target, is expressed in a language that exists at some abstraction level [3]. That is, in forward engineering, MDD can reduce coding burden by generating code from higher-level design models. Likewise, in reverse engineering, design models can be generated from lower-level source code implementations.

The Model-Driven Architecture (MDA) [4] is the OMG's vision of MDD which is built on OMG standards including the Unified Modeling Language (UML) [5], UML Profile, and XML Metadata Interchange (XMI). MDA employs the transformation chain between models of different levels of abstraction: Computation-Independent Model (CIM) which is a high-level model that describes the business context and requirements, Platform-Independent Model (PIM) which describes the structure and behavior of the system independently of any technical platform, Platform-Specific Model (PSM) which is a model that adds technical details to the PIM to map it to a specific platform or technology stack, and Code Model which is represented in a target programming language. The MDA concept of model-to-text transformation enables code generation from PSMs, which can significantly reduce development time despite requiring software engineers to complete the generated code with additional business logic. Conversely, text-to-model transformation creates valuable design documentation that enhances team communication and reduces system comprehension time.

To accelerate web backend development and facilitate understanding of the system, this study focuses on model transformation between PSM models for Node.js and Code models for Node.js in JavaScript language. The contribution of this study consists of (1) UML profile for Node.js which supports PSM-level modeling for web backend and is compatible with both MySQL and MongoDB databases and (2) transformation engines for model-to-code and code-to-model transformation to support forward engineering and reverse engineering, respectively. In an experiment with three case study applications, model-to-code transformation was evaluated in terms of transformation rate when compared with the complete code of the applications. In addition, code-to-model transformation was evaluated in terms of correctness of the PSM models constructed from three implementations of the backend code, i.e. skeleton code generated from model-to-code transformation engine (incomplete code), skeleton code generated from model-to-code transformation engine and enhanced with business logic (complete code), and complete code with further modification to structural code elements (modified code). Note that, although our model-to-code and code-to-model transformation employs OMG standards, i.e. UML Profile and XMI, it should not be considered an MDA approach as not all MDA-related standards are adopted.

Specifically, the OMG's standard language for model transformation, or Query/View/Transformation (QVT), is not adopted for the specification of model transformation. Rather, our approach follows the general concept of MDD and adopts a simplified template-based approach to provide mapping between the PSM-level model and code.

The rest of the paper is organized as follows. Section II discusses background and related work. Section III describes MDD of Node.js backend for both forward and reverse engineering. Section IV presents the experiments to evaluate the MDD approach. The paper concludes in Section V.

II. BACKGROUND AND RELATED WORK

A. Model-Driven Development and UML Profile

In MDD, software development is driven by transformation between software models at different levels of abstraction, where transformation is enabled by transformation tools that recognize transformation rules. These rules define mapping between the metamodels that are used to build the models at any two consecutive levels. In this research, the UML profile [6] is used to extend the UML metamodel to create a platform-specific metamodel for building PSM-level models. A UML profile allows a definition of stereotypes (as extensions to UML metaclasses denoted by << >>), tag definitions (as metaattributes of the extended metaclasses), and constraints. These constructs altogether are used to model software on a specific platform, i.e. Node.js platform in this study.

B. Forward Engineering and Reverse Engineering

Forward engineering [7] is a crucial process in software development which involves transforming informal requirements into a design and a structured program successively. On the contrary, reverse engineering [7] is a critical process in software maintenance and development, particularly for legacy systems. Reverse engineering can be viewed through a transformational lens, where the process involves applying forward engineering transformations in reverse. This means reconstructing a plausible design from the existing code by applying transformation backward to derive a more abstract specification.

C. Node.js and Express.js

Node.js [1] uses Chrome's V8 engine to run JavaScript on the server side, providing a cross-platform, open-source runtime crucial to modern web development. Its single-threaded event loop handles asynchronous I/O operations efficiently, enabling high-concurrency performance without the complexity of thread management. Express.js [8] is a lightweight Node.js framework, providing developers flexibility in selecting libraries and databases. It simplifies web development with a robust routing system that handles various HTTP methods (GET, POST, PUT, DELETE) for RESTful API creation, together with a middleware architecture that processes HTTP requests and responses, enabling authentication, logging, and error handling with custom implementation options.

D. Related Work

There are several approaches to UML-based MDD for web applications and reverse engineering. Rossi [9] introduces a model-driven forward engineering approach to REST API

development by proposing (1) a generic UML-based REST API profile for modeling an API, (2) a RAML-specific profile for modeling an API with an intermediate notation called RAML (or a RESTful API modeling language), and (3) UML-to-RAML transformation rules and a supporting tool. In this approach, several transformation targets can be further supported (such as PHP, Javascript/Node.js, Java/JAX-RS, and so on) if code generators are available and adopted in the tool chain. Chansuwath and Senivongse [10] address a model-driven forward engineering approach for web frontend development using a UML Profile for AngularJS and transformation templates that are used in frontend code generation. Ahmad et al. [11] present a model-driven forward engineering framework for automating MVC-based web application development through a custom UML Profile and a model-to-text transformation engine. Automatic code generation supports implementation of any web application in three MVC-based frameworks, i.e. CodeIgniter, Struts, and ASP.NET. Ullah and Inayat [12] present a model-driven forward engineering approach to template-based automatic code generation for web applications and APIs using class diagram. The generated Python code includes CRUD-based business logic, backend, UIs, object-relational mapping, Routes and APIs. For reverse engineering, Sabir et al. [13] develop a Source to Model Framework (Src2MoF) for generating UML models (class diagram and activity diagram) from Java source code of a legacy system. Their approach uses the proposed Intermediate Model Discoverer (IMD) to parse Java source code to get the intermediate representation of the system and then uses a UML model generator to generate UML models. Rosca and Domingues [14] emphasize the importance of roundtrip engineering as a way to keep several software artifacts up-to-date and consistent in propagating changes between artifacts in multiple directions, i.e. from models to code and vice versa. They compare capabilities of three UML modeling tools, i.e. Papyrus, Modelio, and Visual Paradigm, when performing forward engineering to generate code from UML diagrams, reverse engineering to generate UML diagrams from code, and roundtrip engineering to synchronize UML diagrams and code when each one is modified.

In this study, we adopt mechanisms used by the related work including UML Profile and template-based transformation to support MDD. Unlike related work, we enable MDD for Node.js web backend and support both forward and reverse engineering.

III. MODEL-DRIVEN DEVELOPMENT OF NODE.JS WEB BACKEND

This section presents two key mechanisms for our approach to MDD, i.e. UML Profile for Node.js and template-based transformation. Both are then used to enable model-to-code forward engineering and code-to-model reverse engineering.

A. UML Profile for Node.js

To build a PSM-level model that drives web backend development, we define a UML Profile for Node.js as shown in Fig. 1. The notation of Node.js stereotypes is an extension of the notation of different UML metaclasses, i.e. Package, Class, Dependency, and Operation. Descriptions of the stereotypes are listed in Table I. This UML Profile supports web backend design with specific Node.js components including Route, Controller, Model, and Database. These components follow the separation

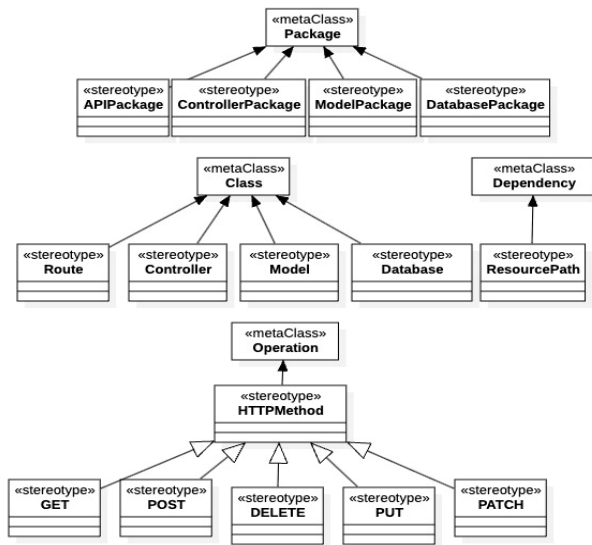


Fig. 1. UML Profile for Node.js.

TABLE I. STEREOTYPES IN UML PROFILE FOR NODE.JS

Stereotype	Description	Metaclass
Route	Component that receives data from the client side to send to the Controller	Class
Controller	The system's logic component	Class
Model	Component that defines the data structure in the database	Class
Database	Used for data storage	Class
ResourcePath	Used with the Route component for sending parameters and defining paths	Dependency
GET	HTTP method for requesting data from the server	Operation
POST	HTTP method for sending data from browser to server to create new data or update existing data on the server	Operation
DELETE	HTTP method for deleting data on the server	Operation
PUT	HTTP method for creating or updating data on the server	Operation
PATCH	HTTP method for partially updating data on the server	Operation
APIPackage	Package that contains Route components	Package
ControllerPackage	Package that contains Controller components	Package
ModelPackage	Package that contains Model components	Package
DatabasePackage	Package that contains Database components	Package

of concerns principle of the Model-View-Controller (MVC) architecture. In web backend development where there is no front-end rendering and the View component can be omitted, the MVC structure still helps separate the backend into Model (data-structure-and-interaction-with-Database layer), Controller (application-logic layer), and Route (HTTP-request-to-Controller-mapping layer). Database supports data storage using both MySQL and MongoDB. For modeling RESTful API of the

backend, the UML Profile supports HTTP methods and ResourcePath that defines a URL segment and parameter for resource access.

B. Template-Based Model-to-Code Transformation

For forward engineering, a number of transformations are defined to map the PSM elements, modeled by the stereotypes, to backend code. We define model-to-code mapping in the form of Node.js code templates as shown in Fig. 2. The case of a car rental system is used as an example. The italicized elements in the code templates correspond to the elements that are extracted from the PSM model. For example, in the Model "CarModel," the attribute "carBrand" and its type "String" are extracted and added in place of $\{attributeName\}$ and $\{attributeType\}$ of $\{templateField\}$ in the code template to generate the field "carBrand" of type "String" in the MongoDB code. (Note that the stereotype Database specified in the PSM is used to select between MongoDB and MySQL code templates.) Fig. 4 depicts our implementation of the model-to-code transformation. First, the UML Profile for Node.js is defined in the StarUML modeling tool [15]. Then, a developer can build a PSM model of a web backend using this profile and export the model to XMI. The XMI representation of the model is imported to the M2C transformation engine. The engine recognizes the M2C mapping, extracts relevant PSM elements to fill in the code templates, and generates the Node.js skeleton code. Finally, the developer can fill in the skeleton code with business logic code to complete the backend code.

C. Template-Based Code-to-Model Transformation

For reverse engineering, a number of transformations are defined to map the code elements to PSM elements, modeled by the stereotypes. We define code-to-model mapping in the form of XMI templates as shown in Fig. 3. The case of a car rental system is used as an example. The italicized elements in the XMI templates correspond to the elements that are extracted from the code. For example, in the MongoDB code, the field "carBrand" and its type "String" are extracted and added in place of $\{model.attribute[.name]\}$ and $\{mappingType(model.attribute[.type])\}$ in the XMI template to generate the content of the XMI tag <ownedAttribute>, which is the XMI representation of the attribute "carBrand" of type "String" in the Model "CarModel." Fig. 5 depicts our implementation of the model-to-code transformation. First, the backend code is imported to the C2M transformation engine and the code is parsed to create an abstract syntax tree (AST) of JavaScript objects. Recognizing the M2C mapping, the engine extracts relevant code elements and fills in the XMI templates to generate the XMI representation of the PSM model of the code using the XML Generator. Finally, the resulting XMI is imported to StarUML for graphical representation of the model.

IV. EVALUATION

In this section, performance evaluation of model-to-code and code-to-model transformation is reported.

A. Performance Evaluation of Model-to-Code Transformation

For forward engineering, PSM-level models of three case studies, i.e. car rental system (CAR), inventory management

Node.js Profile	Model	M2C Mapping (Code Template)	Skeleton Code
<<stereotype>> Route <<stereotype>> ResourcePath		<pre>templateRoute = `router.\${methodNames}/\${path}',   \${controllerName}`; ;`</pre>	<pre>router.get('/cars/getAllCar', getAllCar); router.post('/cars/addCar', addCar); router.get('/cars/car/:carId/getCarById', getCarById); router.patch('/cars/car/:carId/updateCarById', updateCarById); router.put('/cars/car/:carId/replaceCarById', replaceCarById); router.delete('/cars/car/:carId/deleteCarById', deleteCarById);</pre>
<<stereotype>> Database <<stereotype>> Model		<pre>templateField += `.\${attributeName}: {   type: DataTypes.\${attributeType} };`; templateModel = `module.exports =   (sequelize, DataTypes) => {     const \${className} =       sequelize.define('\${className}',         { \${templateField} },         { timestamps: true }       );     return \${className};   };`;</pre>	<pre>module.exports = (sequelize, DataTypes) => { const Car = sequelize.define('Car', { carId: { type: DataTypes.UUID, defaultValue: DataTypes.UUIDV4, primaryKey: true, }, carBrand: { type: DataTypes.STRING }, carRegister: { type: DataTypes.STRING }, carPrice: { type: DataTypes.INTEGER }, carType: { type: DataTypes.STRING }, carTransmission: { type: DataTypes.STRING }, carSeat: { type: DataTypes.STRING }, carStatus: { type: DataTypes.STRING } }, { timestamps: true, }); return Car; };</pre>
<<stereotype>> Database <<stereotype>> Model		<pre>templateField += `.\${attributeName}: { type: \${attributeType}   };`; templateModel = `const mongoose =   require('mongoose'); const \${className} + 'Schema' =   mongoose.Schema(     { \${templateField} }, { timestamps: true }); const \${className} =   mongoose.model('\${className}',     \${className} + 'Schema'); module.exports = \${className};`;</pre>	<pre>const mongoose = require('mongoose'); const carSchema = mongoose.Schema({ carId: { type: UUID }, carBrand: { type: String }, carRegister: { type: String }, carPrice: { type: Number }, carType: { type: String }, carTransmission: { type: String }, carSeat: { type: String }, carStatus: { type: String }, }, { timestamps: true }); const Car = mongoose.model('Car', carSchema); module.exports = Car;</pre>
<<stereotype>> Controller		<pre>templateController = `const   \${operationNames} = asyncHandler(async     (req, res) => { })n'; exportSection =   `module.exports = { \${arrayOfOperationNames} };`;</pre>	<pre>const getAllCar = asyncHandler(async (req, res) => { }); const addCar = asyncHandler(async (req, res) => { }); const getCarById = asyncHandler(async (req, res) => { }); const updateCarById = asyncHandler(async (req, res) => { }); const deleteCarById = asyncHandler(async (req, res) => { }); const replaceCarById = asyncHandler(async (req, res) => { }); module.exports = { getAllCar, addCar, getCarById, updateCarById, deleteCarById, replaceCarById };</pre>

Fig. 2. M2C mapping in the form of code templates.

Code	C2M Mapping (XMI Template)	Model (in XMI)
<pre>router.get('/cars/getAllCar', getAllCar); router.post('/cars/addCar', addCar); router.get('/cars/car/:carId/getCarById', getCarById); router.patch('/cars/car/:carId/updateCarById', updateCarById); router.put('/cars/car/:carId/replaceCarById', replaceCarById); router.delete('/cars/car/:carId/deleteCarById', deleteCarById);</pre>	<pre>let templateRouteXMI = ` <packagedElement xmi:id="\$\${generateId()}" name="\$\${routeName}" xmi:type="uml:Class"> <xmi:Extension extender="StarUML"> <stereotype value="\$\${stereotypeRouteId}"/> </xmi:Extension> <ownedMember xmi:id="\$\${generateId()}" name="\$\${route.dependencyName[.param]}" xmi:type="uml:Dependency" client="\$\${route.dependencyName[.from]}" supplier="\$\${route.dependencyName[.to]}" <xmi:Extension extender="StarUML"> <stereotype value="\$\${stereotypeResourcePATHId}"/> </xmi:Extension> </ownedMember> <ownedOperation xmi:id="\$\${generateId()}" name="\$\${route.controllerName[.]}" xmi:type="uml:Operation"> <xmi:Extension extender="StarUML"> <stereotype value="\$\${stereotypeHTTPMethodId}"/> </xmi:Extension> </ownedOperation> </packagedElement></pre>	<pre><packagedElement xmi:id="CarsRouteID" name="CarsRoute" xmi:type="uml:Class"> <xmi:Extension extender="StarUML"> <stereotype value="stereotypeRouteId"/> </xmi:Extension> <ownedMember xmi:id="unique_id" name="/carId" xmi:type="uml:Dependency" client="CarsRouteId" supplier="CarRouteId"> <xmi:Extension extender="StarUML"> <stereotype value="stereotypeResourcePATHId"/> </xmi:Extension> </ownedMember> <ownedOperation xmi:id="unique_id" name="getAllCar" xmi:type="uml:Operation"> <xmi:Extension extender="StarUML"> <stereotype value="stereotypeHTTPMethodId"/> </xmi:Extension> </ownedOperation> ... </packagedElement> <packagedElement xmi:id="CarRouteId" name="CarRoute" xmi:type="uml:Class"> <xmi:Extension extender="StarUML"> <stereotype value="stereotypeRouteId"/> </xmi:Extension> <ownedOperation xmi:id="unique_id" name="getCarById" xmi:type="uml:Operation"> <xmi:Extension extender="StarUML"> <stereotype value="stereotypeHTTPMethodId"/> </xmi:Extension> </ownedOperation> ... </packagedElement></pre>
<pre>const mongoose = require('mongoose'); const carSchema = mongoose.Schema({ carId: { type: UUID }, carBrand: { type: String }, carRegister: { type: String }, carPrice: { type: Number }, carType: { type: String }, carTransmission: { type: String }, carSeat: { type: String }, carStatus: { type: String }, }, { timestamps: true }); const Car = mongoose.model('Car', carSchema); module.exports = Car;</pre>	<pre>let templateModelXMI = `<packagedElement xmi:id="\$\${generateId()}" name="\$\${model.model[0]}" xmi:type="uml:Class"> <xmi:Extension extender="StarUML"> <stereotype value="\$\${stereotypeModelId}"/> </xmi:Extension> <ownedAttribute xmi:id="\$\${generateId()}" name="\$\${model.attribute[.name]}" type="\$\${mappingType(model.attribute[.type])}" visibility="private" xmi:type="uml:Property"/> </packagedElement></pre>	<pre><packagedElement xmi:id="CarModelId" name="CarModel" xmi:type="uml:Class"> <xmi:Extension extender="StarUML"> <stereotype value="stereotypeModelId"/> </xmi:Extension> <ownedAttribute xmi:id="unique_id" name="carId" type="Uuid_id" visibility="private" xmi:type="uml:Property"/> <ownedAttribute xmi:id="unique_id" name="carBrand" type="String_id" visibility="private" xmi:type="uml:Property"/> <ownedAttribute xmi:id="unique_id" name="carRegister" type="String_id" visibility="private" xmi:type="uml:Property"/> <ownedAttribute xmi:id="unique_id" name="carPrice" type="Number_id" visibility="private" xmi:type="uml:Property"/> ... </packagedElement></pre>
<pre>const getAllCar = asyncHandler(async (req, res) => { }); const addCar = asyncHandler(async (req, res) => { }); const getCarById = asyncHandler(async (req, res) => { }); const updateCarById = asyncHandler(async (req, res) => { }); const deleteCarById = asyncHandler(async (req, res) => { }); const replaceCarById = asyncHandler(async (req, res) => { }); module.exports = { getAllCar, addCar, getCarById, updateCarById, deleteCarById, replaceCarById };</pre>	<pre>let templateControllerXMI = ` <packagedElement xmi:id="\$\${generateId()}" name="\$\${controller.className[0]}" xmi:type="uml:Class"> <xmi:Extension extender="StarUML"> <stereotype value="\$\${stereotypeControllerId}"/> </xmi:Extension> <ownedOperation xmi:id="\$\${generateId()}" name="\$\${controller.controllerName[.]}" xmi:type="uml:Operation"/> </packagedElement></pre>	<pre><packagedElement xmi:id="CarControllerId" name="CarController" xmi:type="uml:Class"> <xmi:Extension extender="StarUML"> <stereotype value="stereotypeControllerId"/> </xmi:Extension> <ownedOperation xmi:id="unique_id" name="getAllCar" xmi:type="uml:Operation"> <ownedOperation xmi:id="unique_id" name="addCar" xmi:type="uml:Operation"/> <ownedOperation xmi:id="unique_id" name="getCarById" xmi:type="uml:Operation"/> ... </packagedElement></pre>

Fig. 3. C2M mapping in the form of XMI templates.

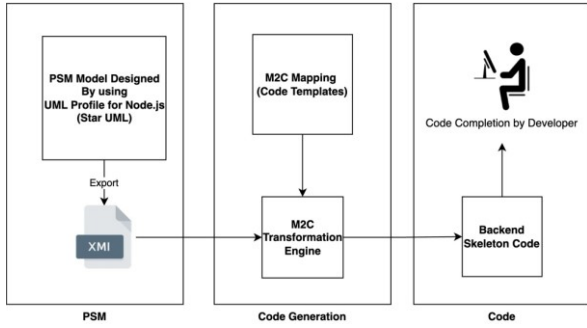


Fig. 4. Model-to-code transformation.

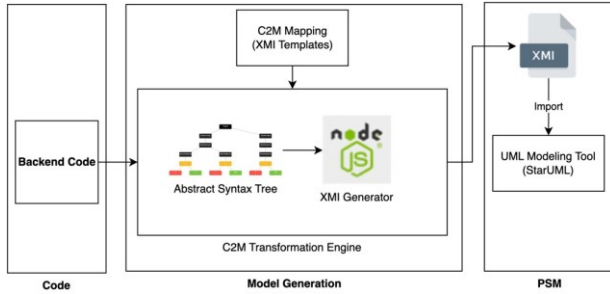


Fig. 5. Code-to-model transformation.

system (INV), and learning management system (LMS), were used in the performance evaluation of model-to-code transformation. In the car rental system (CAR), a renter selects a car from a rental service provider. The renter specifies the pickup and return date, time, and location. The system calculates the rental cost and issues the bill. The renter then proceeds to make the payment to complete the booking process. In the inventory management system (INV), a merchant places a purchase order to buy products from a supplier. The supplier issues an invoice, and the merchant proceeds with the payment. Once the payment is confirmed, the supplier makes a product delivery. In the learning management system (LMS), an instructor can create and manage an online course by uploading the course description, materials, and videos, and setting course pricing. A learner can make payment to enroll and access the course content. The system also tracks the learner's progress throughout the course.

Information about the PSM models of the three case studies is shown in Table II. The performance metric is the transformation rate, i.e. the percentage of skeleton code (in LOC) which is automatically generated compared with the complete code after a developer has filled in the business logic. The results in Table III show high transformation rates for Model (86.18-94.53%) and Route (70.73-90.52%). For Controller, the transformation rate is lower (12.36-30.52%) as expected, because most business logic code is part of Controller. The performance is consistent across MySQL and MongoDB implementations.

B. Performance Evaluation of Code-to-Model Transformation

For reverse engineering, backend code of three case studies, i.e. car rental system (CAR), inventory management system

TABLE II. DETAILS OF PSMs OF CASE STUDY SYSTEMS

System	PSM Element	NOP ^a	NOC ^a	NOM ^a	NOA ^a
CAR	Controller	1	9	38	0
	Route	1	17	38	0
	Model	1	6	0	38
INV	Controller	1	7	33	0
	Route	1	12	33	0
	Model	1	8	0	41
LMS	Controller	1	4	16	0
	Route	1	7	16	0
	Model	1	8	0	27

^a NOP = number of packages, NOC = number of classes, NOM = number of methods, NOA = number of attributes.

TABLE III. PERFORMANCE OF M2C TRANSFORMATION

System	DB	Transformation Rate (%)		
		Controller	Route	Model
CAR	MongoDB	30.52	90.52	94.53
	MySQL	16.75	90.52	91.76
INV	MongoDB	23.80	90.41	93.83
	MySQL	21.41	90.41	86.28
LMS	MongoDB	14.33	70.73	86.18
	MySQL	12.36	70.73	89.62

(INV), and learning management system (LMS), were used in the performance evaluation of code-to-model transformation. For each case study, three implementations of backend code were used: (1) skeleton code generated from M2C transformation engine (i.e. incomplete code), (2) skeleton code generated from M2C transformation engine and enhanced with business logic (i.e. complete code), and (3) complete code with further modification to structural code elements by adding more Controller, Route, and Model to the code (i.e. modified code). Information about these implementations is shown in Table IV. The performance metric is the transformation correctness, i.e. the percentage of stereotype elements that are generated from corresponding code elements. The results in Table V show 100% correctness, as corresponding PSM elements were generated from all code elements.

V. CONCLUSION

The MDD of Node.js-based backend addresses key challenges in web application development through automated code generation and model reconstruction. The implementation of a specialized UML Profile for Node.js, combined with bidirectional transformation engines, demonstrate significant benefits in development efficiency and system maintenance. The high transformation rate, especially for Model and Route, in model-to-code transformation helps increase productivity in backend development. The perfect transformation correctness in

TABLE IV. DETAILS OF THREE IMPLEMENTATIONS OF CASE STUDY SYSTEMS

System	Code Element	Incomplete Code and Complete Code				Modified Code			
		<i>NOP^b</i>	<i>NOC^b</i>	<i>NOM^b</i>	<i>NOF^b</i>	<i>NOP^b</i>	<i>NOC^b</i>	<i>NOM^b</i>	<i>NOF^b</i>
CAR	Controller	1	9	38	0	1	10	38	0
	Route	1	17	38	0	1	18	38	0
	Model	1	6	0	38	1	7	0	43
	Database	1	1	0	1	1	1	0	1
INV	Controller	1	7	33	0	1	8	38	0
	Route	1	12	33	0	1	13	38	0
	Model	1	8	0	41	1	9	0	49
	Database	1	1	0	1	1	1	0	1
LMS	Controller	1	4	16	0	1	5	20	0
	Route	1	7	16	0	1	8	20	0
	Model	1	8	0	27	1	9	0	33
	Database	1	1	0	1	1	1	0	1

^b. NOP = number of packages, NOC = number of classes, NOM = number of methods, NOF = number of fields.

TABLE V. PERFORMANCE OF C2M TRANSFORMATION

System	DB	Transformation Correctness (%)			
		<i>Controller</i>	<i>Route</i>	<i>Model</i>	<i>Database</i>
CAR	MongoDB	100	100	100	100
	MySQL	100	100	100	100
INV	MongoDB	100	100	100	100
	MySQL	100	100	100	100
LMS	MongoDB	100	100	100	100
	MySQL	100	100	100	100

code-to-model transformation ensures that consistent design documentation is maintained with the code. Future directions of

this research involve extending the MDD approach to include frontend components and improving Controller transformation rates through enhanced business logic templates. Qualitative evaluation, such as user studies or developer feedback, that could highlight ease of use, learnability, and integration potential of this MDD approach is worth exploring.

REFERENCES

- [1] "Node.js." nodejs.org. Accessed: Apr. 30, 2025. [Online]. Available: <https://nodejs.org/en>
- [2] "2023 developer survey." stackoverflow.co. Accessed: Apr 30, 2025. [Online]. Available: <https://survey.stackoverflow.co/2023/>
- [3] S. J. Mellor, A. N. Clark, and T. Futagami, "Guest editors' introduction: Model-driven development" IEEE Softw., 20(5), pp. 14-18, Sep./Oct. 2003, doi: 10.1109/MS.2003.1231145.
- [4] "MDA specifications." omg.org. Accessed: Apr. 30, 2025. [Online]. Available: <https://www.omg.org/mda/specs.htm>
- [5] "Documents associated with Unified Modeling Language (UML) version 2.5." omg.org. Accessed: Apr. 30, 2025. [Online]. Available: <http://www.omg.org/spec/UML/2.5/>
- [6] "UML Profile." omg.org. Accessed: Apr. 30, 2025. [Online]. Available: <https://www.omg.org/mda/specs.htm>
- [7] I. D. Baxter and M. Mehlich, "Reverse engineering is reverse forward engineering," Sci. Comput. Program., vol. 36, issue 2-3, pp. 131-147, Mar. 2000, doi: 10.1016/S0167-6423(99)00034-9.
- [8] "Express.js." expressjs.com. Accessed: Apr. 30, 2025. [Online]. Available: <https://expressjs.com/>
- [9] D. Rossi, "UML-based model-driven REST API development," in Proc. 12th Int. Conf. Web Inf. Syst. Technol. – Vol. 1 (WEBIST), 2016, pp. 194-201.
- [10] W. Chansuwath and T. Senivongse, "A model-driven development of web applications using AngularJS framework," in Proc. 2016 IEEE/ACIS 15th Int. Conf. Comput. Inf. Sci. (ICIS 2016), Okayama, Japan, Jun. 26-29, 2016, pp. 683-688, doi: 10.1109/ICIS.2016.7550838.
- [11] S. Ahmad, T. Rana, and A. Maqbool, "A model-driven framework for the development of MVC-based (web) application," Arabian J. Sci. Eng., vol. 47, pp. 1733-1747, 2022, doi: 10.1007/s13369-021-06087-4.
- [12] I. Ullah and I. Inayat, "Template-based automatic code generation for web application and APIs using class diagram," in Proc. 2022 Int. Conf. Frontiers Inf. Technol. (FIT), Islamabad, Pakistan, Dec. 12-13, 2022, pp. 332-337, doi: 10.1109/FIT57066.2022.00067.
- [13] U. Sabir, F. Azam, S. Haq, M. Anwar, W. Haider, and A. Amjad, "A model driven reverse engineering framework for generating high level UML models from Java source code," IEEE Access, vol. 7, 2019, pp. 158931-158950, doi: 10.1109/ACCESS.2019.2950884.
- [14] D. Rosca and L. Domingues, "A systematic comparison of roundtrip software engineering approaches applied to UML class diagram," Procedia Comput. Sci., vol. 181, pp. 861-868, 2021, doi: 10.1016/j.procs.2021.01.240.
- [15] "StarUML." staruml.io. Accessed: Apr. 30, 2025. [Online]. Available: <https://staruml.io/>